

JARVIS

Linux Cluster Monitoring Business Requirements

**Prepared by,
Zeeshan Khateeb
Business Systems Analyst
03/02/2026**

Contents

Executive Summary	4
Project Objectives	5
Needs Statement.....	5
Scope	5
Business Requirements	6
Data Model and Key Metrics	7
Performance Requirements	8
Security Requirements.....	8
Stakeholders	8
Schedule, Timeline, and Milestones.....	9
Cost-Benefit Analysis.....	9
Success Metrics / Key Performance Indicators (KPI's)	10

Assumptions and Constraints	10
Risks and Mitigations.....	11
Acceptance Criteria	11
Glossary	11

Executive Summary

The Jarvis Linux Cluster Administration (LCA) team manages a Linux cluster composed of multiple Rocky Linux servers connected through an internal network. To support capacity planning and operational stability, the team requires a centralized way to capture hardware specifications and track resource usage trends over time. At present, collecting and analyzing this information manually is time-consuming and does not scale as the cluster grows.

This project proposes a Linux Cluster Monitoring Agent that collects system-level metrics from each host and stores them in a centralized PostgreSQL database. The solution enables administrators and engineering users to query usage data using SQL and generate reporting insights for resource planning, troubleshooting, and operational decision-making. The monitoring workflow is automated through scheduled execution so that usage data is continuously captured without manual intervention as shown in the overview diagram.

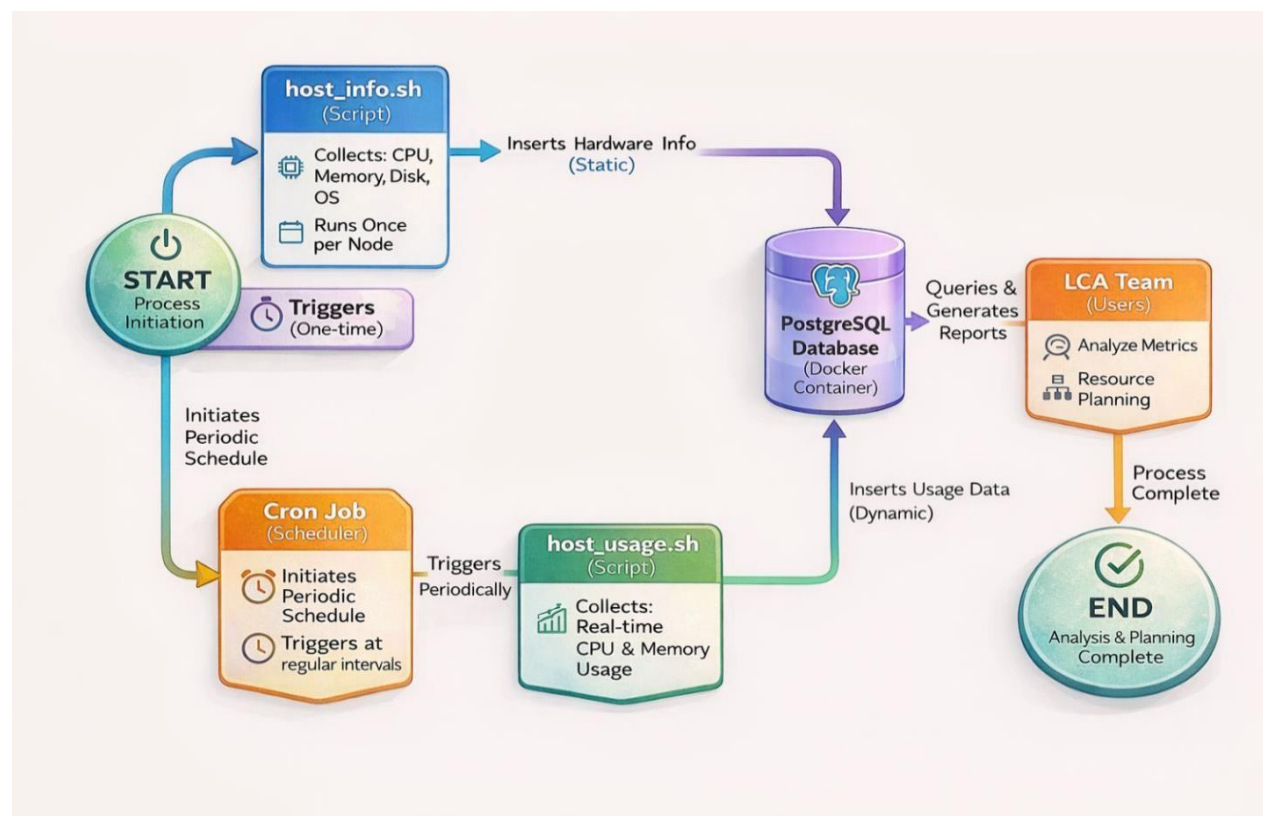


Figure 1: Overview Diagram

Project Objectives

The objective of this project is to provide the LCA team with reliable visibility into cluster hardware profiles and ongoing resource utilization. The monitoring agent must collect static host specifications once per host and capture dynamic usage metrics at regular intervals. The collected data must be stored centrally in a structured database so that users can run SQL queries to answer business questions such as average memory usage per node over time, CPU utilization trends, and capacity constraints.

A successful outcome is a working minimum viable product (MVP) that demonstrates end-to-end data collection, storage, and query capability, and that can be deployed in a cluster environment with minimal changes. The solution should be simple to operate, easy to validate, and suitable for ongoing use as a learning and monitoring foundation.

Needs Statement

The LCA team needs consistent and accurate hardware usage reporting to make informed decisions about server capacity and cluster scaling. Without a centralized monitoring approach, administrators must log into individual servers and run commands manually, which is inefficient, error-prone, and difficult to standardize across many nodes. In addition, a lack of historical usage data reduces the team's ability to identify trends and predict when resource limits will be reached.

The monitoring agent addresses these needs by collecting standardized metrics from each server and storing them in a shared PostgreSQL repository. This enables the LCA team to view current utilization patterns and analyze historical usage, improving planning accuracy and reducing the operational effort required to monitor cluster resources.

Scope

In scope, the solution will provision a centralized PostgreSQL database using Docker and provide Bash-based scripts that collect host information and host usage metrics from Linux servers. Static host information, such as CPU architecture and total memory, will be collected once per host and inserted into the database. Dynamic usage information, such as memory availability, CPU utilization, disk I/O, and disk space, will be collected at a regular interval and inserted continuously to build a history of resource usage.

Out of scope, the MVP will not implement a full alerting and notification system, advanced visualization dashboards, or commercial monitoring tooling integrations. The project will not introduce complex access management or enterprise identity integrations beyond what is required to run scripts and access the database in a controlled environment. These enhancements may be considered in future iterations after the MVP is validated.

Business Requirements

The Linux Cluster Monitoring Agent must satisfy the following business requirements:

BR-1: Automated Hardware Data Collection

The system must automatically collect static host information including CPU architecture, number of cores, total memory, and host identifier for each node in the Linux cluster.

BR-2: Continuous Resource Monitoring

The system must collect dynamic usage metrics including CPU usage, memory usage, disk I/O statistics, and available disk space at regular intervals.

BR-3: Centralized Data Storage

All collected monitoring data must be stored in a centralized PostgreSQL database to allow consistent querying and reporting.

BR-4: Historical Data Availability

The system must store historical resource usage data to support trend analysis and capacity planning over time.

BR-5: Automated Data Collection Scheduling

The monitoring scripts must run automatically using cron scheduling with a configurable interval (default: 1 minute).

BR-6: Data Query Capability

Users must be able to query monitoring data using SQL to generate reports about system utilization and performance trends.

BR-7: System Reliability

The monitoring solution must ensure that data collection failures are logged and detectable through system logs or database verification queries.

The system must provide a repeatable process to capture and persist host hardware and usage metrics for a Linux cluster. It must support a multi-host environment where each node runs a

lightweight agent and sends metrics to a centralized database. The data must be stored in a structured format that supports SQL-based reporting and analytics.

The solution must include a script that collects static host information and inserts it into the database, and a script that collects ongoing usage information and inserts it on a recurring schedule. The recurring schedule must be automated so that usage data is captured consistently, for example by using cron at a one-minute interval. The database must support queries that allow users to measure utilization levels per host and over time, enabling planning decisions such as when to add or remove servers.

From an operational perspective, the system must be easy to start and stop, and must provide verification that data has been inserted successfully. The solution must be maintainable within a standard Git-based workflow and be suitable for use in a Rocky Linux environment.

Data Model and Key Metrics

The monitoring system stores data in a PostgreSQL database composed of two primary tables: **host_info** and **host_usage**.

host_info_table

Stores static hardware details for each server in the cluster.

Key attributes include:

- host_id (Primary Key)
- hostname
- cpu_number
- cpu_architecture
- total_memory
- timestamp

host_usage_table

Stores dynamic resource usage metrics collected at regular intervals.

Key attributes include:

- usage_id (Primary Key)
- host_id (Foreign Key)
- cpu_usage_percentage
- memory_usage_percentage
- disk_io
- disk_available
- timestamp

Key metrics monitored include:

- CPU utilization
- Memory usage
- Disk I/O

- Available disk space
- Host hardware specifications

These metrics allow administrators to monitor cluster health, analyze trends, and identify potential capacity constraints.

Performance Requirements

The monitoring system must meet the following performance requirements:

- The system must support monitoring of **multiple Linux hosts simultaneously**.
- Data collection scripts must execute within **5 seconds per host**.
- The database must support insertion of monitoring records **every minute without data loss**.
- Queries retrieving monitoring data must return results within **2 seconds for standard analytical queries**.

Security Requirements

The monitoring solution must ensure the following security measures:

- Database access must require **authenticated credentials**.
- Only authorized users should be able to query or modify monitoring data.
- Monitoring scripts must run with **least privilege permissions**.
- Sensitive system information must not be exposed outside the internal cluster network.
- Network communication between hosts and database must occur within a **secured internal network environment**.

Stakeholders

The primary stakeholder is the Jarvis Linux Cluster Administration (LCA) team, which uses the collected metrics to monitor server health and make capacity planning decisions. The LCA team defines what metrics are needed, how frequently usage should be collected, and what reporting outputs are valuable for operational planning.

Supporting stakeholders include the implementation team responsible for building and validating the monitoring scripts and database setup, as well as users who may write SQL queries to generate reports and answer business questions from the collected data. A support or operations function may also participate in deploying the solution and maintaining it over time, including confirming cron scheduling and monitoring the agent's ongoing execution.

Schedule, Timeline, and Milestones

Phase	Activity	Duration
Requirements & Design	Define monitoring metrics and database schema	2 days
Development	Implement host_info.sh and host_usage.sh scripts	3 days
Database Setup	Deploy PostgreSQL container and create tables	1 day
Testing	Validate data collection and database insertion	2 days
Automation	Configure cron scheduling	1 day
Documentation & Demo	Prepare documentation and project demonstration	1 day

*Total estimated project duration: **10 days***

The project will proceed in phases that align with a simplified SDLC workflow: design, implementation, testing, and deployment demonstration. During the design phase, requirements are confirmed including target metrics, collection frequency, and storage approach. During implementation, the PostgreSQL database is provisioned and scripts are developed to collect and insert host information and usage data. During testing, scripts are executed manually to validate expected outputs and inserts, and the recurring schedule is validated through cron logs. During deployment, the scripts and database setup are demonstrated in a clean runbook-style flow.

A key milestone is completing the MVP capable of collecting and persisting both host_info and host_usage data and verifying the data through SQL queries. Another milestone is demonstrating continuous automated data capture through cron, supported by logs that confirm recurring successful inserts.

Cost-Benefit Analysis

Cost Component	Estimated Cost
Development Time	40 hours
Infrastructure (Docker/PostgreSQL)	Minimal (open source)
Operational Maintenance	Low

The primary cost of the current state is the manual effort required to collect hardware specifications and usage data across multiple servers. Manual monitoring does not scale well as

server count increases and increases the risk of inconsistent data collection. Another cost is delayed decision-making when historical utilization data is unavailable, which can lead to inefficient over-provisioning or performance issues caused by under-provisioning.

The benefits of the monitoring agent include reduced manual effort, standardized metric collection, improved visibility into current and historical resource usage, and faster, more confident capacity planning decisions. By storing metrics centrally, the team can answer key operational questions using SQL without repeatedly accessing each server, improving efficiency and enabling data-driven planning.

Success Metrics / Key Performance Indicators (KPI's)

The success of the Linux Cluster Monitoring Agent will be evaluated using the following measurable indicators:

1. Data Collection Reliability

The monitoring agent should successfully collect and insert host information and usage metrics for all nodes in the cluster without failures during scheduled execution.

2. Monitoring Coverage

The system should support monitoring of all servers participating in the Linux cluster and ensure that each host reports metrics to the centralized database.

3. Data Availability for Analysis

Collected monitoring data should be available for SQL-based queries, allowing administrators to retrieve CPU, memory, and disk usage metrics for analysis and reporting.

4. Automation Effectiveness

The monitoring scripts should run automatically through cron scheduling without requiring manual intervention after deployment.

5. Reduction in Manual Monitoring Effort

The monitoring solution should reduce the need for administrators to manually log into individual servers to collect system metrics.

6. Historical Data Tracking

The system should maintain historical monitoring data to allow trend analysis and support long-term capacity planning.

Assumptions and Constraints

It is assumed that the Linux cluster nodes can communicate internally and that required network connectivity to the database host is available. It is also assumed that the target environment uses Rocky Linux or a compatible Linux distribution that supports the commands required for metric collection. The solution assumes that PostgreSQL can be provisioned via Docker and that the team is permitted to run scheduled tasks using cron.

A key constraint is that this project focuses on an MVP and therefore prioritizes data collection

and storage over advanced alerting, visualization, or enterprise-grade security controls. The design also aims to remain lightweight and easy to deploy, which limits scope to Bash scripting and a centralized database rather than additional services.

Risks and Mitigations

A risk is that metric collection may differ across environments if dependencies or Linux command outputs vary. This risk is mitigated by using widely available Linux CLI tools and validating script execution during testing. Another risk is incomplete data capture if cron is misconfigured or scripts fail silently. This is mitigated by writing outputs to logs and validating recurring inserts through database queries and log review.

There is also a risk related to database availability, because a centralized database becomes a dependency for data capture. This risk is mitigated by ensuring the database container can be started reliably and by validating connectivity as part of the runbook steps.

Acceptance Criteria

The BRD is considered successful when it clearly documents the business need, scope, stakeholders, and requirements for the Linux Cluster Monitoring Agent in a professional and readable format. The monitoring solution is considered MVP-complete when it provisions PostgreSQL via Docker, creates required tables, inserts `host_info` data successfully at least once per host, and inserts `host_usage` data successfully on a recurring schedule. SQL queries must be able to retrieve stored metrics to answer utilization-related business questions.

The solution must demonstrate automated execution through cron and provide verification evidence through logs and database records. Documentation must be sufficiently clear that a reviewer can understand what is being built, why it is needed, and what success looks like.

Glossary

“LCA” refers to the Linux Cluster Administration team. “Host” refers to a Linux server/node participating in the cluster. “Agent” refers to the Bash scripts that collect and insert metrics. “MVP” refers to the minimum viable product implementation that demonstrates end-to-end functionality.